ASSIGNMENT 8: OPTIMIZE ANYTHING

The 14-Day Vibe-Coding Challenge — Comprehensive Technical Report

Student Name: Ayaan Mirza	**Roll No:** 241070043
Course: Optimization Techniques (Lab 8)	**Submission:** Public GitHub Repository & Technical Report
Methodology: Mandatory Vibe Coding / AI-Assisted Development	**Verification:** Pytest Automated Suite (14/14 Passing)

Executive Summary

This report documents the completion of **Assignment 8: The 14-Day Vibe-Coding Challenge** in Optimization Techniques. Optimization is approached here not as isolated software coding, but as a formal, measurable problem-solving discipline. Across ten compulsory experiment tracks (**Tracks A to J**), real and simulated computational processes were formulated into decision variables, objective functions, and constraints. Each process was benchmarked against a naive or unoptimized baseline, optimized using methods from Chapter 6, and validated with reproducible empirical measurements and strict correctness checks.

Every algorithm specified in Section 5—including continuous first- and second-order methods, decoupled backpropagation, discrete branch-and-bound and ILP, QUBO/Ising models, quantum algorithms (QAOA, VQE, QFT, Shor, Grover, Annealing), submodular facility coverage, parallel speedup laws, federated learning (FedAvg), multi-task MGDA, compiler passes, operating system schedulers, and database index knapsacks—has been implemented from first principles and experimentally benchmarked.

Key Empirical Results Across All 10 Tracks

Track	Domain / Method	Baseline Metric	Optimized Metric	Measured Improvement	Correctness Verification
Track A	Continuous & Curvature	GD: 78 iterations to tol	Newton: 1 iteration; Adam: stable on noise	Machine precision on convex; robust on saddle	Decoupled analytical backprop verified
Track B	Combinatorial & QUBO	Random assignment: Cut=4.5	Exact / SA / Tabu: Cut=**9.0**	Global optimum cut achieved; 0% gap in ILP	Exact exhaustive search baseline
Track C	Quantum QAOA & VQE	Classical exact cut: 9.0	QAOA expected cut: **8.46**	**0.94 approximation ratio**; VQE converged	Statevector unitarity & shot convergence
Track D	Submodular & Parallel	Static load balance: 1.62	Work-stealing: **1.08**; FedAvg E=5	Greedy coverage **0.91** \ge (1 - 1/e) \approx 0.632	Zero submodularity violations in 100 trials
Track E	Green Datacenter Case Study	First-Fit: 76.84, 118.4 kg CO_2	GA: **49.33, 74.1 kg CO_2**	**35.8% cost cut, 37.4% carbon cut**	**0 SLA violations**; hard constraints satisfied
Track F	Compiler Passes & RegAlloc	Dynamic Insts: 32, Cycles: 53	Dynamic Insts: **20**, Cycles: **32**	**37.5% instruction reduction, 39.6% cycle cut**	Byte-identical output `[40, 6550]`
Track G	xv6 OS Scheduling	Round-Robin resp: 2.3 ticks	MLFQ resp: **0.2 ticks**	**91.3% interactive response speedup**	xv6 boots cleanly; user benchmarks pass
Track H	DBMS Workload Indexing	Table Scans: 4.38 ms	Index Seeks: **1.39 ms** (Q1: 0.01 ms)	**3.15x overall speedup (165.4x on Q1)**	Query results 100% byte-identical
Track I	Quantum Algorithms & QML	Classical search: O(N) queries	Grover: O(\sqrt{N}) queries	Asymptotic quantum query speedup proved	7 algorithms verified; honest negative sim result
Track J	Software Eng (1,011 LOC)	All tests: 2062.2 ms	ILP Minimized: **285.0 ms**	**86.2% CI test time reduction**	**100% branch coverage (50/50)** preserved

1. Track-by-Track Technical Breakdown & Experimental Outcomes

Track A: Continuous Optimization & Curvature

- **Artifacts:** Code: [landscapes.py](tracks/track_a_continuous/landscapes.py), [optimizers.py](tracks/track_a_continuous/optimizers.py), [neural_net.py](tracks/track_a_continuous/neural_net.py) | Results: [benchmark_summary.csv](tracks/track_a_continuous/results/benchmark_summary.csv), [track_a_convergence_plots.png](tracks/track_a_continuous/results/track_a_convergence_plots.png)
- **Objective:** Minimize mathematical loss landscapes with differing curvature properties (strongly convex quadratic, non-convex Rosenbrock valley, and non-convex saddle-point functions), and decouple backpropagation from optimizer updates in a 2-layer Neural Network.
- **Formulation:**

min_{\theta} L(\theta) \quad \text{s.t.} \quad \theta \in \mathbb{R}^D

Continuous algorithms tested: Gradient Descent (GD), Polyak Momentum, Nesterov Accelerated Gradient (NAG), AdaGrad, RMSProp, Adam, and Newton-Raphson with Levenberg-Marquardt damping (H + \mu I).

- **Baseline vs Optimized Findings:**
- On the strongly convex quadratic (\kappa = 15), Newton-Raphson reached machine precision (10^{-14}) in 1 iteration, while standard GD required 78 iterations.
- On the saddle point landscape (f(x_1, x_2) = x_1^2 - x_2^2 + 0.1 x_2^4), standard GD stalled in the zero-gradient neighborhood, while Adam and Momentum used accumulated directional velocity to escape along the negative curvature eigenvector.
- On noisy gradients (\sigma = 0.5), Adam maintained the lowest final objective variance (0.041 \pm 0.008) compared to GD (0.382 \pm 0.114).
- **Backpropagation Decoupling:** In `SmallMLP`, `backward(y)` computes pure analytical gradients \nabla_W L via the multivariable chain rule into `grads` without altering parameter values, proving architectural independence from the optimizer step.

Track B: Discrete Optimization & Combinatorial Formulations

- **Artifacts:** Code: [combinatorial_solvers.py](tracks/track_b_discrete/combinatorial_solvers.py), [qubo_formulation.py](tracks/track_b_discrete/qubo_formulation.py), [classical_qubo_solvers.py](tracks/track_b_discrete/classical_qubo_solvers.py) | Results: [knapsack_benchmark.csv](tracks/track_b_discrete/results/knapsack_benchmark.csv), [maxcut_qubo_benchmark.csv](tracks/track_b_discrete/results/maxcut_qubo_benchmark.csv), [track_b_discrete_plots.png](tracks/track_b_discrete/results/track_b_discrete_plots.png)
- **Objective:** Solve 0-1 resource allocation and combinatorial graph partitioning, establishing exact baselines for quantum benchmarks.
- **Formulation:**
- **0-1 Knapsack:** \max \sum v_i x_i s.t. sum w_i x_i \le W, x_i \in \{0, 1\}. Solved via Greedy, Dynamic Programming (O(NW)), Branch-and-Bound with fractional LP relaxation, and 0-1 ILP.
- **Graph Max-Cut QUBO / Ising:** Maximize cut weight on an 8-node graph by minimizing E(x) = x^T Q x or Ising Hamiltonian H = \frac{1}{2}\sum w_{ij} s_i s_j with spin conversion s_i = 1 - 2x_i.
- **Constrained Feature Selection:** Formulated with quadratic cardinality penalty \lambda (\sum x_i - K)^2.
- **Measured Results:**
- Knapsack: DP, B&B, and ILP reached the exact global optimum (V=220.0), while Greedy executed in 0.04 ms with an optimality gap of only 3.8%.
- Max-Cut: Exact exhaustive search identified the maximum cut weight of **9.0** (optimal bitstring `01010101`). Classical Simulated Annealing and Tabu Search also converged to cut weight 9.0 in 45 ms and 18 ms.
- Cardinality Penalty: At \lambda \ge 5.0, 100% of optimal assignments strictly obeyed the cardinality constraint sum x_i = 3.

Track C: Quantum Optimization (QAOA, VQE & Hardware Limits)

- **Artifacts:** Code: [qaoa.py](tracks/track_c_quantum/qaoa.py), [vqe.py](tracks/track_c_quantum/vqe.py), [simulator.py](tracks/track_c_quantum/simulator.py), [gradients.py](tracks/track_c_quantum/gradients.py), [barren_plateau.py](tracks/track_c_quantum/barren_plateau.py) | Results: [qaoa_vs_classical.csv](tracks/track_c_quantum/results/qaoa_vs_classical.csv), [barren_plateau_scaling.csv](tracks/track_c_quantum/results/barren_plateau_scaling.csv), [track_c_quantum_plots.png](tracks/track_c_quantum/results/track_c_quantum_plots.png)
- **Objective:** Solve Track B's Max-Cut QUBO on a statevector simulator using QAOA, solve ground-state energy with VQE, and measure the real physical constraints of NISQ devices.
- **Algorithms:** QAOA (p=2 alternating layers), VQE (hardware-efficient parameterized ansatz), Parameter-Shift Rule, SPSA, and Quantum Natural Gradient.
- **Measured Results:**
- QAOA achieved an **expected cut of 8.46** and sampled maximum cut of **9.0**, attaining a **0.94 approximation ratio** relative to the classical exact optimum.
- Parameter-Shift vs SPSA: Parameter-shift produced deterministic monotonic convergence; SPSA required 65% fewer circuit evaluations per step but exhibited stochastic oscillation.
- **Hardware Noise:** Single-qubit depolarizing error rate degraded expected cut from 8.46 (p=0) down to 5.82 (p=0.12), illustrating that quantum advantage vanishes rapidly without fault-tolerant quantum error correction.
- **Barren Plateaus:** Measured gradient variance decay across N \in \{2, 4, 6, 8\} qubits: variance decayed exponentially from 0.125 (N=2) to 0.0019 (N=8), empirically validating McClean et al.'s Barren Plateau theorem:

$$\text{Var}_{\theta}\left[\frac{\partial \langle H \rangle}{\partial \theta_k}\right] \in O(2^{-N})$$

Track D: Submodular, Parallel & Distributed Optimization

- **Artifacts:** Code: [submodular.py](tracks/track_d_parallel_distributed/submodular.py), [parallel_runtime.py](tracks/track_d_parallel_distributed/parallel_runtime.py), [federated.py](tracks/track_d_parallel_distributed/federated.py), [multi_task.py](tracks/track_d_parallel_distributed/multi_task.py) | Results: [parallel_scaling.csv](tracks/track_d_parallel_distributed/results/parallel_scaling.csv), [federated_benchmark.csv](tracks/track_d_parallel_distributed/results/federated_benchmark.csv), [track_d_plots.png](tracks/track_d_parallel_distributed/results/track_d_plots.png)
- **Objective:** Demonstrate submodular diminishing returns, parallel speedup laws, work-stealing load balancing, federated learning (FedAvg), and multi-task gradient conflict resolution.
- **Measured Results:**
- **Submodularity:** Facility location coverage verified diminishing returns $f(A \cup \{e\}) - f(A) \ge f(B \cup \{e\}) - f(B)$ across 100 randomized subset trials with **0 violations**. Greedy selection achieved an empirical approximation ratio of **0.91**, easily surpassing the theoretical lower bound of $(1 - 1/e) \approx 0.632$.
- **Parallel Scaling:** Heterogeneous numerical workloads executed across 1, 2, 4, and 8 worker processes. Empirical speedup scaled to $4.18\times$ on 8 workers. Amdahl's Law accurately modeled the serial bottleneck plateau for parallel fraction $f = 0.88$ ($S_{\text{max}} = 8.33$), while Gustafson's Law modeled scaled speedup under expanding problem sizes.
- **Work Stealing:** Blumofe-Leiserson work-stealing reduced worker load imbalance ratio from 1.62 (static chunking) down to 1.08.
- **Federated Learning:** FedAvg trained on 5 non-IID client partitions. Increasing local epochs from E=1 to E=5 reduced required communication rounds by 60% with identical convergence loss (0.342).
- **Multi-Task Optimization:** On competing gradient directions, Multiple-Gradient Descent Algorithm (MGDA) found the minimum-norm Pareto consensus point, preventing negative task transfer seen under naive equal weighting.

Track E: Main Case Study — Green Datacenter Workload & Energy-Carbon Optimization

- **Artifacts:** Code: [problem_formulation.py](tracks/track_e_case_study/problem_formulation.py), [solvers.py](tracks/track_e_case_study/solvers.py), [strategy_table.py](tracks/track_e_case_study/strategy_table.py) | Results: [case_study_summary.csv](tracks/track_e_case_study/results/case_study_summary.csv), [track_e_case_study_plots.png](tracks/track_e_case_study/results/track_e_case_study_plots.png) | Dedicated Case Study: CASE_STUDY.md
- **Problem Statement:** 24-hour scheduling of compute workloads across heterogeneous server clusters under dynamic Time-of-Use electricity tariffs (\ / kWh) and time-varying grid carbon emissions (gCO_2 / kWh with solar peaks).
- **Mathematical Formulation:**

$$\min \sum_{t=1}^{24} \left[\left(\text{Tariff}(t) + \beta \cdot \text{Carbon}(t)\right) \sum_{j=1}^M P(u_{jt}) \right] + \sum_{i=1}^N \lambda \cdot \max(0, \text{finish}_i - \text{deadline}_i)$$

Subject to: Server CPU/RAM capacities, job arrival releases, hard completion deadlines, and quadratic power scaling $P(u) = P_{\text{idle}} + (P_{\text{max}} - P_{\text{idle}}) u^2$.

- **Experimental Comparison:**
- **Baseline Naive First-Fit:** Schedules tasks immediately upon arrival on the first available server. Total Operational Cost: **\76.84**, Carbon Emissions: **118.4 kg CO_2**.
- **Green Carbon-Aware Greedy:** Evaluates marginal cost across all feasible start windows [a_i, d_i - w_i] to align heavy jobs with low-tariff, high-solar hours. Total Operational Cost: **\54.91 (28.5% cost reduction)**, Carbon Emissions: **81.5 kg CO_2 (31.2% carbon abatement)**.
- **Genetic Algorithm (GA):** Population-based metaheuristic with tournament selection and SLA repair. Total Operational Cost: **\49.33 (35.8% cost reduction)**, Carbon Emissions: **74.1 kg CO_2 (37.4% carbon abatement)**, with **0 SLA violations**.

Track F: Compiler Optimization Passes & ILP Register Allocation

- **Artifacts:** Code: [ir.py](tracks/track_f_compiler/ir.py), [passes.py](tracks/track_f_compiler/passes.py), [code_gen.py](tracks/track_f_compiler/code_gen.py), [reg_alloc.py](tracks/track_f_compiler/reg_alloc.py) | Results: [optimization_pipeline_summary.csv](tracks/track_f_compiler/results/optimization_pipeline_summary.csv), [track_f_compiler_plots.png](tracks/track_f_compiler/results/track_f_compiler_plots.png)
- **Objective:** Construct a Three-Address Code (TAC) intermediate representation compiler pipeline with passes that provably optimize instruction count and cycle times without altering semantics.
- **Passes Implemented:**

1. Constant Folding & Forward Propagation

2. Common Subexpression Elimination (CSE)

3. Dead Code Elimination (DCE)

4. Loop-Invariant Code Motion (LICM)

- **Register Allocation:** Formulated as Graph Colouring on variable interference graphs, comparing Greedy Chaitin-Briggs Kempe heuristic against exact 0-1 ILP minimizing spill penalties.
- **Verification & Gains:**
- Dynamic instructions executed reduced from **32 to 20 (37.5% reduction)**; estimated CPU cycle count dropped from **53 to 32 (39.6% reduction)**.
- **Correctness Check:** Output log of unoptimized program `[40, 6550]` and optimized program `[40, 6550]` were **byte-for-byte identical**.

Track G: xv6 Operating-System Scheduler Optimization

- **Artifacts:** Code: [emulator.py](tracks/track_g_xv6/emulator.py), [xv6_mlfq_priority.patch](tracks/track_g_xv6/xv6_src/xv6_mlfq_priority.patch), [cow_fork.patch](tracks/track_g_xv6/xv6_src/cow_fork.patch) | Results: [xv6_scheduler_benchmarks.csv](tracks/track_g_xv6/results/xv6_scheduler_benchmarks.csv), [track_g_xv6_plots.png](tracks/track_g_xv6/results/track_g_xv6_plots.png)

- **Kernel Concepts Optimized:** Preemptive CPU scheduling, process starvation, and virtual memory page sharing.
- **Kernel Features Added:**
- Multilevel Feedback Queue (MLFQ) with 3 priority levels (Q_0: 1 tick, Q_1: 4 ticks, Q_2: 8 ticks), demotion upon quantum exhaustion, periodic priority boost (every 100 ticks) to guarantee starvation freedom, and a `sys_setpriority` system call.
- Copy-On-Write (COW) Fork in virtual memory kernel sources (`kalloc.c`, `vm.c`).
- **Benchmarking across 3 Workloads (CPU-Bound, I/O Bursty, Mixed):**
- On Mixed Workload: Default Round-Robin resulted in an average interactive response time of 2.3 ticks; MLFQ reduced this to **0.2 ticks (91.3% improvement)** while keeping turnaround times balanced.
- Stride scheduling achieved exact proportional-share CPU allocations matching ticket assignments with a Jain's fairness index of 0.548.

Track H: DBMS Optimization & Index Selection Knapsack

- **Artifacts:** Code: [schema_and_data.py](tracks/track_h_dbms/schema_and_data.py), [index_optimizer.py](tracks/track_h_dbms/index_optimizer.py) | Results: [index_selection_solvers.csv](tracks/track_h_dbms/results/index_selection_solvers.csv), [query_latency_and_plans.csv](tracks/track_h_dbms/results/query_latency_and_plans.csv), [track_h_dbms_plots.png](tracks/track_h_dbms/results/track_h_dbms_plots.png)
- **Objective:** Optimize a synthetic TPC-H database workload on SQLite by selecting secondary indexes under a strict storage budget B = 200 KB.
- **Mathematical Formulation:** 0-1 Knapsack maximizing aggregate latency reduction sum \Delta L_i x_i subject to sum S_i x_i \le B, x_i \in \{0, 1\}.
- **Experimental Findings:**
- Without secondary indexes, SQLite executed sequential table scans (`SCAN lineitem`, `SCAN orders`), requiring **4.38 ms** for the workload.
- Exact ILP Knapsack selected optimal composite index pair `idx_lineitem_ship_disc` and `idx_orders_custkey`.
- Executed `ANALYZE;` to update SQLite's internal statistics catalog.
- Query plan transformed to index seeks (`SEARCH lineitem USING INDEX idx_lineitem_ship_disc`), slashing Q1 latency by **165.4x** and overall workload execution time from **4.38 ms to 1.39 ms (3.15x speedup / 68.2% latency reduction)**.
- **Strict Correctness Check:** All query result sets before and after optimization matched 100% identically.

Track I: Quantum Algorithms Suite & QML Advantage Reality Check

- **Artifacts:** Code: [algorithms.py](tracks/track_i_quantum_advantage/algorithms.py), [qml_classifier.py](tracks/track_i_quantum_advantage/qml_classifier.py), [complexity_comparison.py](tracks/track_i_quantum_advantage/complexity_comparison.py) | Results: [quantum_algorithms_suite.json](tracks/track_i_quantum_advantage/results/quantum_algorithms_suite.json), [grover_vs_linear_search.csv](tracks/track_i_quantum_advantage/results/grover_vs_linear_search.csv), [qml_vs_classical.csv](tracks/track_i_quantum_advantage/results/qml_vs_classical.csv), [track_i_quantum_plots.png](tracks/track_i_quantum_advantage/results/track_i_quantum_plots.png)
- **Quantum Algorithms Implemented from Scratch:**

1. *Deutsch-Jozsa:* 1-query deterministic classification of constant vs balanced functions.

2. *Bernstein-Vazirani:* 1-query exact recovery of hidden bitstring (`1011`).

3. *Grover's Search:* Amplified target state in \lfloor \frac{\pi}{4}\sqrt{N} \rfloor queries with 94.8% success probability.

4. *Quantum Fourier Transform (QFT):* Verified exact unitarity F^\dagger F = I and matched classical FFT.

5. *Quantum Phase Estimation (QPE):* Resolved eigenphase \theta = 0.375 with 3 bits of precision.

6. *Shor's Factoring (Order-Finding):* Extracted period r=4 for 7^r \equiv 1 \pmod{15}, factoring N=15 into primes \{3, 5\}.

7. *Quantum Annealing Simulator:* Demonstrated transverse-field quantum tunneling across spin barriers.

- **QML vs Classical MLP Benchmark:**

- Variational Quantum Classifier (VQC with angle embedding) achieved 87.5% accuracy but required 600 circuit evaluations and 485 ms.
- Classical 2-Layer MLP achieved 92.5% accuracy in 14.2 ms (34.1\times faster).
- **The Honest Negative Result:** While Grover demonstrates quadratic asymptotic query advantage (O(\sqrt{N}) vs O(N)), simulating quantum statevectors on classical computers incurs an exponential memory and matrix-vector multiplication overhead of O(2^N), resulting in classical simulation runtimes that were up to **225\times slower** than classical linear search. Real quantum advantage requires physical, fault-tolerant QPU hardware with thousands of physical qubits.

Track J: Software-Engineering Optimization on Non-Trivial Codebase

- **Artifacts:** Code: [test_optimizer.py](tracks/track_j_software_engineering/test_optimizer.py), [codebase/](tracks/track_j_software_engineering/codebase/) | Results: [test_optimization_summary.csv](tracks/track_j_software_engineering/results/test_optimization_summary.csv), [track_j_software_engineering_plots.png](tracks/track_j_software_engineering/results/track_j_software_engineering_plots.png)
- **Codebase Scale:** Engineered an 8-module analytics and data processing infrastructure totaling **1,011 Lines of Code** (surpassing the 1,000 LOC requirement).
- **Problem Formulation:** Test-Suite Minimization (TSM) formulated as 0-1 ILP Set Cover:

$$\min_{x} \sum_{i=1}^{40} t_i x_i \quad \text{s.t.} \quad \sum_{i: b \in B_i} x_i \ge 1 \quad \forall b \in \{1, \dots, 50\}, \quad x_i \in \{0, 1\}$$

- **Measured Results:**
- Unoptimized execution of all 40 tests required **2062.2 ms**.
- Exact ILP minimization selected the minimal test subset required to cover all 50 code branches, reducing test execution time to **285.0 ms (86.2% time savings)**.
- **100% Branch Coverage Guaranteed:** All 50/50 branches remained covered.
- Test Prioritization (TSP) via greedy APFD ordered tests to maximize fault detection velocity, achieving an APFD score of 0.657.

---

2. Compulsory Final Questions (Section 11 Answers)

Q1: What process did you choose, and what evidence showed that it needed optimization?

For the main case study (Track E), we chose **Sustainable Cloud Datacenter Workload Allocation & Dynamic Energy-Carbon Cost Management**. Datacenters currently account for over 1.5% of global electricity consumption. Profiling the baseline first-fit scheduling process revealed two severe inefficiencies:

1. Servers remained powered on at idle power (80\text{W} per server) even during low utilization.

2. Batch compute tasks were dispatched immediately upon submission, coincidentally colliding with peak evening electricity tariffs (\0.28/\text{kWh}) and peak carbon grid intensity (450\,\text{gCO}_2/\text{kWh}), generating \76.84 in cost and 118.4 kg of CO_2 emissions over 24 hours.

Q2: How did you formulate the objective function and constraints?

The problem was formulated as a mixed-integer non-linear optimization program over time slots $t \in \{0, \dots, 23\}$:

$$\min_{x, y, f} \sum_{t=0}^{23} \left[\left(\text{Tariff}(t) + \beta \cdot \text{Carbon}(t)\right) \sum_{j=1}^M \left(y_{jt} P_{\text{idle}} + (P_{\text{max}} - P_{\text{idle}}) u_{jt}^2 \right) \right] + \sum_{i=1}^N \mu \cdot \max(0, \text{finish}_i - \text{deadline}_i)$$

Constraints:

- Job release times: $\text{start}_i \ge \text{arrival}_i$
- Server capacity: $\sum_{i \in \text{active}(j, t)} \text{RAM}_i \le \text{CapRAM}_j$ and $\sum \text{CPU}_i \le \text{CapCPU}_j$
- Non-preemptive execution: tasks run contiguously for their required duration.

Q3: Which Chapter 6 methods did you test, and why were they appropriate for their respective experiments?

We tested the entire Chapter 6 methods portfolio:

- **Continuous (GD, Momentum, NAG, AdaGrad, RMSProp, Adam, Newton, Gauss-Newton):** Appropriate for continuous parameter optimization and neural network training where analytical gradients exist.
- **Combinatorial (Greedy, DP, Branch-and-Bound, ILP):** Appropriate for knapsack resource allocation, compiler register allocation, DBMS index selection, and CI test suite minimization where solutions are discrete subsets.
- **QUBO / Ising / Quantum (QAOA, VQE, Annealing):** Appropriate for NP-hard graph partitioning (Max-Cut) and physical spin Hamiltonians.
- **Submodular Greedy:** Appropriate for sensor coverage and facility location exhibiting diminishing returns.
- **Distributed (FedAvg, MGDA, GradNorm):** Appropriate for multi-client privacy-preserving learning and conflicting multi-task gradient directions.

Q4: How did the optimized solution compare with the baseline?

In every track, optimized methods delivered measurable improvements over baselines:

- **Track E (Datacenter):** Genetic Algorithm reduced total cost by **35.8%** and carbon emissions by **37.4%** with 0 SLA violations.
- **Track F (Compiler):** Optimization passes reduced dynamic instruction count by **37.5%** and cycles by **39.6%**.
- **Track G (xv6 OS):** MLFQ reduced interactive response time by **91.3%** under mixed workloads.
- **Track H (DBMS):** ILP index selection delivered a **3.15x overall speedup** (Q1 speedup of **165.4x**).
- **Track J (Software Eng):** ILP test minimization reduced test suite runtime by **86.2%** while guaranteeing 100% branch coverage.

Q5: What trade-offs, failures, or limitations did you discover?

1. Accuracy vs Speed: In Track B Knapsack, Greedy ran in 0.04 ms with a 3.8% optimality gap, whereas exact DP and ILP took longer but guaranteed 0% gap.

2. Quantum Simulation Overhead (The Honest Negative Result): While quantum Grover search achieves theoretical O(\sqrt{N}) query complexity, simulating it on a classical CPU is O(2^N) in time and space, proving that quantum algorithms cannot provide speedups via simulation.

3. Barren Plateaus: Deep parameterized quantum circuits suffer exponentially vanishing gradients, meaning random initialization fails on large qubit registers without specialized layerwise pre-training.

4. Local Epochs in FedAvg: Increasing local epochs E reduces communication bandwidth but introduces client drift under highly non-IID partitions.

Q6: What evidence demonstrates that your results are correct and reproducible?

- All 14 automated test cases in `tests/test_all_tracks.py` pass (`14 passed in 0.74s`).
- Deterministic pseudo-random seeds (`seed=42`) were fixed across all synthetic datasets and benchmark runs.
- **Strict Equivalence Checks:** Track F compiler outputs (`[40, 6550]`) and Track H DBMS query result rows matched 100% identically between unoptimized and optimized code.
- All experiment scripts write raw CSVs and high-resolution PNG plots into reproducible directory structures.

Q7: How could the optimization approach be adapted if the scale, data, constraints, or hardware changed?

- If job count in Track E grows to N \ge 100,000, the Genetic Algorithm should be transitioned to streaming Green Greedy or distributed Ray/Dask workers.
- If storage budget in Track H increases, multi-column indexes covering join predicates can be incorporated.
- If physical quantum hardware (e.g. IBM Quantum QPU) becomes available, error mitigation techniques (Zero-Noise Extrapolation, dynamical decoupling) must be added to counteract the gate depolarizing noise measured in Track C.

Q8: For the compiler, xv6, and DBMS tracks, which optimization did you implement, what OS or database concept did you learn from it, and how did you show that correctness was preserved?

- **Compiler (Track F):** Implemented Constant Folding, CSE, DCE, LICM, and ILP Register Allocation. Learned how liveness analysis constructs interference graphs. Correctness was proven by executing unoptimized and optimized TAC on a Virtual Machine and asserting identical output sequences.
- **xv6 OS (Track G):** Implemented MLFQ and Stride scheduling in kernel space with periodic priority boosting, plus COW fork. Learned how kernel context switching and lock contention affect fairness and starvation. Verified that xv6 boots cleanly and user benchmarks complete with correct process termination.
- **DBMS (Track H):** Implemented 0-1 Knapsack index selection on SQLite. Learned how SQLite's cost-based query planner relies on `ANALYZE;` catalog statistics to choose index seeks over full table scans. Correctness was verified by asserting that every query's returned records were byte-identical before and after index creation.

Q9: What advantages and limits of QML, quantum algorithms, and parallel computing did your experiments reveal compared with classical computing?

- **Parallel Computing:** Delivers immediate, practical speedup on classical hardware, bounded by Amdahl's Law serial fraction 1 - f. Work-stealing prevents straggler cores.
- **Quantum Algorithms:** Offer proven asymptotic speedups for specific mathematical structures (Grover O(\sqrt{N}), Shor O((\log N)^3) polynomial factoring).
- **Quantum Limits:** NISQ quantum computing is severely limited by: (1) exponential simulation overhead on classical computers, (2) gate depolarizing noise which flattens expectation landscapes, (3) Barren Plateaus where gradient variance decays as O(2^{-N}), and (4) input data encoding bottlenecks (e.g. angle embedding) which make QML classifiers much slower than classical MLPs on tabular data.

Q10: Which AI tools did you use, how did their outputs differ, and which AI-generated errors did you find and fix?

- **Tools Used:** Antigravity IDE (Gemini 3.8 Flash), Claude Code (Claude 3.5 Sonnet), and ChatGPT (GPT-4o).
- **Differences:** Gemini 3.8 Flash was fastest and best integrated with local files and multi-step tasks. Claude 3.5 Sonnet excelled at deep mathematical code analysis and compiler graph interference logic.
- **AI Errors Found & Fixed:**

1. *SciPy MILP Bounds Format Error:* AI generated a list of tuples for `milp(bounds=...)`; fixed by importing `scipy.optimize.Bounds(lb=0, ub=1)`.

2. *Federated Broadcasting Mismatch:* AI subtracted 2D label column `(N, 1)` from 1D probability vector `(N,)`; fixed using `y.ravel()`.

3. *DBMS Index Selection without ANALYZE:* AI created SQLite indexes without running `ANALYZE;`, causing the query planner to ignore indexes; fixed by adding `ANALYZE;`, yielding a **3.15x speedup**.

---

3. AI Tools Register & Vibe Coding Failure Log Summary

AI Tools Register

AI Tool / Assistant	Model Version	Experiment Tracks Covered	Dates Active	Nature of Work Performed
Antigravity IDE Assistant	Gemini 3.8 Flash	Tracks A to J, Core, Tests, Demo	2026-10-29 – 2026-10-31	Architectural formulation, code drafting, unit tests, automated reports
Claude Code	Claude 3.5 Sonnet	Track F (Compiler), Track I (Quantum), Track G (xv6)	2026-10-29	Kempe coloring audit, Shor order-finding verification, xv6 lock inspection

AI Tool / Assistant	Model Version	Experiment Tracks Covered	Dates Active	Nature of Work Performed
ChatGPT (OpenAI)	GPT-4o	Track E (Green Datacenter), Track H (DBMS)	2026-10-29	Power model formulation check, TPC-H index knapsack formulation validation

Documented AI Failure Log

1. Track B (SciPy MILP Bounds Crash): AI generated `bounds=[(0, 1) for _ in range(N)]` which caused `ValueError: bounds must be convertible into an instance of scipy.optimize.Bounds`. Fixed by importing and instantiating `scipy.optimize.Bounds(lb=0.0, ub=1.0)`. Lesson: API signatures in modern numerical libraries change; verify against installed documentation.

2. Track D (Federated Binary Cross-Entropy Shape Mismatch): AI subtracted 2D label vector `(N, 1)` from 1D probability vector `(N,)`, creating an unexpected `(N, N)` matrix during NumPy broadcasting. Fixed via `y.ravel()`. Lesson: Matrix calculus prompts require defensive shape checks.

3. Track H (DBMS Index Selection Ignored without ANALYZE): AI generated SQLite secondary indexes without issuing `ANALYZE;`. SQLite's cost-based query planner defaulted to table scans, giving 0.79x speedup. Fixed by adding `ANALYZE;`, providing SQLite with table cardinality statistics and enabling index seeks for a 3.15x speedup. Lesson: Database query planners rely on statistical metadata catalogs.

---

4. Conclusion & Vibe Coding Reflections

Summary of Findings

Assignment 8 demonstrates that optimization is not simply about writing faster code—it requires systematically identifying inefficiencies, establishing rigorous baselines, formulating decision variables and constraints, selecting appropriate algorithmic strategies from Chapter 6, and defending the results with reproducible empirical evidence. Using AI as a collaborative vibe-coding partner accelerated development while requiring vigilant human verification to catch subtle numerical, API, and architectural failures.

Possible Next Steps & Future Extensions

1. Physical QPU Hardware Execution: Execute the Track C QAOA and Track I Grover algorithms on real physical QPUs (e.g., IBM Quantum via Qiskit Runtime) and benchmark Zero-Noise Extrapolation (ZNE) and readout error mitigation against our measured depolarizing noise simulations.

2. Distributed Cluster Scheduling: Scale Track E's datacenter scheduler from N=40 synthetic batch jobs to N \ge 100,000 distributed Ray/Kubernetes tasks, implementing real-time streaming Green Greedy with distributed battery energy storage (BESS) integration.

3. Compiler Polyhedral Loop Transformations: Extend Track F's TAC pipeline with polyhedral loop tiling, loop fusion, and SIMD vectorization passes to target modern multicore AVX-512 and GPU tensor cores.

4. Machine-Learning Powered DBMS Index Selection: Integrate deep reinforcement learning or learned cost models into Track H's index selector to dynamically adapt to non-stationary OLTP/OLAP hybrid workloads.